

Realtek8762E 模组快速开始指导

V 1.0.3
2026/02/11

修订历史（Revision History）

日期	版本	修改	作者	Reviewer
2023/11/23	V 1.0.0	初版	袁佳威	
2024/02/26	V 1.0.1	新增 Mesh2.0 功能说明	苏杭	
2024/05/23	V 1.0.2	更新厂包/烧录相关说明	苏杭	
2026/02/11	V 1.0.3	更新环境安装命令	马腾飞	

目录

1. 概述	5
2. 搭建开发编译环境	5
2.1. 系统要求	5
2.2. 环境安装	5
3. 代码下载	5
3.1. 代码权限申请	5
3.2. 代码下载	5
4. 代码目录	5
5. 代码编译	6
5.1. 配置	6
5.2. 编译	6
5.3. 清空	7
6. 模组烧录	7
6.1. 准备工作	7
6.1.1. 烧录工具	7
6.1.2. 模组开发板	7
6.1.3. 开发调试待烧录的文件	7
6.2. 烧录步骤	8
7. 调试 LOG	9
8. 制作 OTA 包	10
9. 制作厂包	10
10. 分区表	13
11. MESH 色温灯工程	13
11.1. 宏定义配置	13
11.2. MESH 状态回调	13
11.3. OTA 状态回调	15
11.4. 设备信息回调	16
11.5. MESH 消息回调	17
11.5.1. 读取属性回调 <i>do_spec_get</i>	17
11.5.2. 设置属性回调 <i>do_spec_set</i>	18
11.5.3. 执行方法回调 <i>do_spec_invoke</i>	19
11.5.4. 属性操作接口定义	20
11.6. MESH 属性上报	22
11.6.1. 属性上报 <i>miio_mesh_properties_changed</i>	22
11.6.2. 属性周期上报 <i>miio_mesh_properties_period_publish_init</i>	22
11.7. MESH 事件上报	23
11.7.1. 事件触发上报 <i>miio_mesh_event_occurred</i>	23
11.8. 服务器信息请求	24
11.8.1. 请求服务器信息 <i>int miio_mesh_request_property (uint8_t type)</i>	24
11.9. MESH2.0 功能	24

11.9.1. 秒级配网 (Beta)	24
11.9.2. 远程 OTA.....	24
11.9.3. 一致性场景 (Beta)	25
11.10. 私有广播发送	25
11.11. 设备重置	26
12. 单火开关工程	27
12.1. 宏定义配置	27
12.2. 上电功耗	27
12.3. 固件搬运功耗	28
12.4. 扫描窗口	28
12.5. DLPS 回调	29

1. 概述

本文档旨在指导开发者快速熟悉 Realtek8762E 芯片平台(MHCB12G/MHCB12S)模组软件开发环境，文档中包含开发环境搭建、示例工程介绍、模组烧录调试等内容。

2. 搭建开发编译环境

2.1. 系统要求

推荐使用 Ubuntu 20.04 LTS。

2.2. 环境安装

Ubuntu 环境下命令行顺序执行下列命令安装所需环境：

```
sudo dpkg --add-architecture i386
sudo apt-get -y update
ls
sudo apt-get -y install autoconf automake bison build-essential default-jdk dfu-util genromfs flex git gperf
sudo apt-get -y install repo
sudo apt install python-pip

sudo apt-get -y install lib32ncurses5-dev libc6-dev-i386 libx11-dev libx11-dev:i386 libxext-dev
sudo apt-get -y install libxext-dev:i386 net-tools pkgconf unionfs-fuse zlib1g-dev
sudo apt-get -y install g++-8 g++-8-multilib
sudo apt-get -y install libpulse-dev:i386
sudo apt-get -y install libasound2-dev:i386
sudo apt-get -y install libasound2-plugins:i386
sudo apt-get -y install libusb-1.0-0-dev
sudo apt-get -y install libusb-1.0-0-dev:i386
sudo apt install gcc-arm-none-eabi
sudo apt install kconfig-frontends
```

3. 代码下载

3.1. 代码权限申请

在小米 IoT 开发者平台上申请 BLE MESH SDK 权限，选择对应模组型号。

3.2. 代码下载

仓库地址：<https://partner-gitlab.mioffice.cn/miot/bsp/miot-rtl8762e-b12g-sdk>

下载命令：`git clone --recursive git@partner-gitlab.mioffice.cn:miot/bsp/miot-rtl8762e-b12g-sdk.git -b master`

4. 代码目录

1. 根目录结构：

```
$ tree . -L 1
```

```
.
├── apps
├── build.sh -> nuttx/tools/build.sh
├── external
├── frameworks
├── nuttx
├── prebuilts
├── tests
└── vendor
```

2. Vendor 目录结构：

```
$ tree vendor/ -L 1
```

```
vendor/
├── Kconfig
```

```

|—— Make.defs
|—— Makefile
|—— realtek
|—— sim
|—— xiaomi

```

3. mijia_ble_mesh 目录结构:

```
$ tree vendor/xiaomi/mijia_ble_mesh/ -L 2
```

```

vendor/xiaomi/mijia_ble_mesh/
|—— CMakeLists.txt
|—— Kconfig
|—— Make.defs
|—— Makefile
|—— prj.conf
|—— src
    |—— libmesh_auth_vela_rtl8762e.a
    |—— libmesh_auth_vela_rtl8762e_debug.a
    |—— main.c
    |—— main.h
    |—— main_lpn.c
    |—— mesh_vnd_df.h
    |—— mesh_vnd_lpn.h
    |—— mible.h
    |—— mible_crypto.c
    |—— mible_crypto.h
    |—— mible_dfu.h
    |—— mible_mesh.h
    |—— mible_spec.h
    |—— mible_user_api.c
    |—— mible_user_api.h
    |—— mi_crypto_backend_mbedtls.c
    |—— mi_crypto_backend_mbedtls.h
    |—— miio_user_api.h
    |—— mi_mesh_otp.c
    |—— mi_mesh_otp.h
    |—— model.c
    |—— nvm.c
    |—— third_party

```

5. 代码编译

5.1. 配置

```
./build.sh vendor/realtek/boards/rtl8762e/configs/app menuconfig
```

注意：非必要不执行，使用默认配置即可。

5.2. 编译

现有两个默认编译配置。

常电版本编译命令：

```
./build.sh vendor/realtek/boards/rtl8762e/configs/app
```

低功耗版本编译命令：

```
./build.sh vendor/realtek/boards/rtl8762e/configs/app_pm
```

注意：编译厂包必须单线程，不能加 -jxx（多线程编译固件内容不一致）。

编译生成 app image 位与 vendor\realtek\tools\bee\application_is\Debug\bin 目录下。

5.3. 清空

```
./build.sh vendor/realtek/boards/rtl8762e/configs/app distclean
```

```
或./build.sh vendor/realtek/boards/rtl8762e/configs/app_pm distclean
```

注意：修改配置或头文件后，需先 distclean 后再编译，根据选择编译配置执行 distclean 命令。

6. 模组烧录

烧录操作切勿进行全擦除，否则证书等重要信息丢失，模组不可用。

6.1. 准备工作

6.1.1. 烧录工具

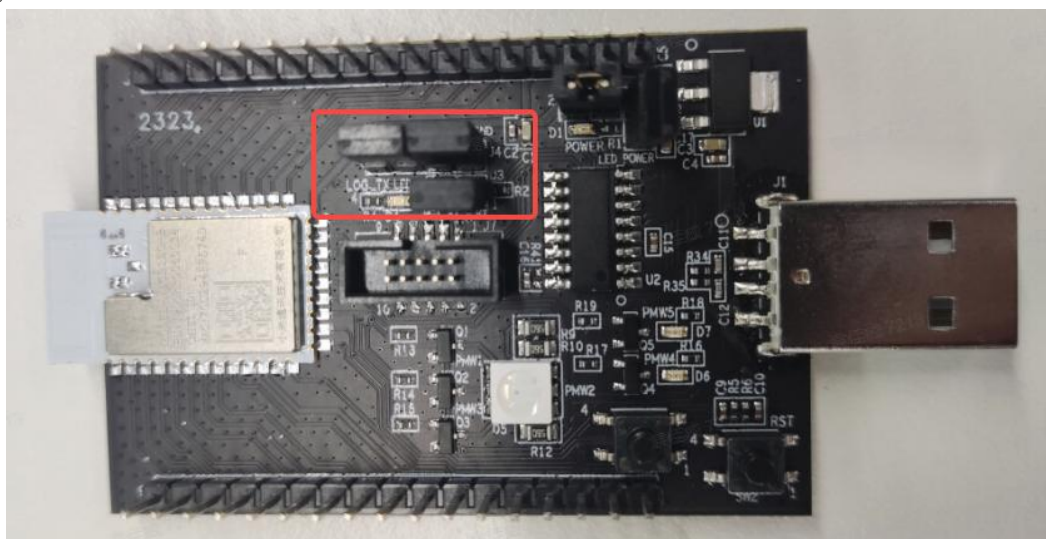
烧录使用 MPTool 工具烧录，MPTool 支持两种烧录模式：量产模式和调试模式。量产模式用于产线生产，调试模式用于开发人员开发调试时使用。调试模式默认是关闭的，开发者需要运行“Registry Set.exe”工具打开该功能。

MPTool 工具位于代码目录 vendor/xiaomi/mijia_ble_mesh/tools, 工具详细介绍请参考工具目录下 Doc 文档。

***注意：量产模式用于产线生产，但为避免工厂量产时产生误操作，请勿在产线量产的电脑上运行“Registry Set.exe”。**

6.1.2. 模组开发板

烧录前模组需进入烧录模式，将开发板 J3、J4 处插针依次两两短接，连接好后复位模组即可进入烧录模式。



6.1.3. 开发调试待烧录的文件

在使用小米 IoT 模组开发调试时，只需要将 flash 分区表、app image 和 patch 这三个文件烧录至小米 IoT 模组，选择下表中必选文件烧录即可，其他文件不用烧录。

文件名	功能	默认位置	必选
flash_map.ini	Flash Layout	vendor\xiaomi\mijia_ble_mesh\tools\download	M
OTAHeader_Bank0_1.0.0.1-813f61fba2a9ef54bdffb8ef6d595640.bin	Bank0 OTA Header File	vendor\xiaomi\mijia_ble_mesh\tools\download	
fsbl_mp_master##_1.1.0.0_dea0248d-818a246824b295b221dda5e3cc347abd.bin	Bank0 Secure Boot Loader Image	vendor\xiaomi\mijia_ble_mesh\tools\download	
Patch_ext_MP-7f52202a23c41ea26bd2b257dae2925.bin	Bank0 APP Date1 File	vendor\xiaomi\mijia_ble_mesh\tools\download	M
XIAOMI_Patch_MP_master_1.0.352.0_10845ded-7788e7a5f1bba2442aa7251677ca1e27.bin	Bank0 ROM Patch Image	vendor\xiaomi\mijia_ble_mesh\tools\download	M
application_is_MP_sdk_1.1.0.3_1201a39e-fdb7091227e61a26294ff4e3b3f600db.bin	Bank0 APP Image	编译生成的 app image，位于目录 vendor\realtek\tools\bee\application_is\Debug\bin	M
configFile_2023.09.18.10-2055ef9a6dfea005687062faa972cbbb.bin	System Config File	vendor\xiaomi\mijia_ble_mesh\tools\download	

***注意：configFile 不能随意烧写，会改变模组 MAC 地址与射频标定。**

***注意：工程中提供了两版 Patch，单火开关应使用 XIAOMI_SLOW_COPY_Patch 以避免固件搬运电**

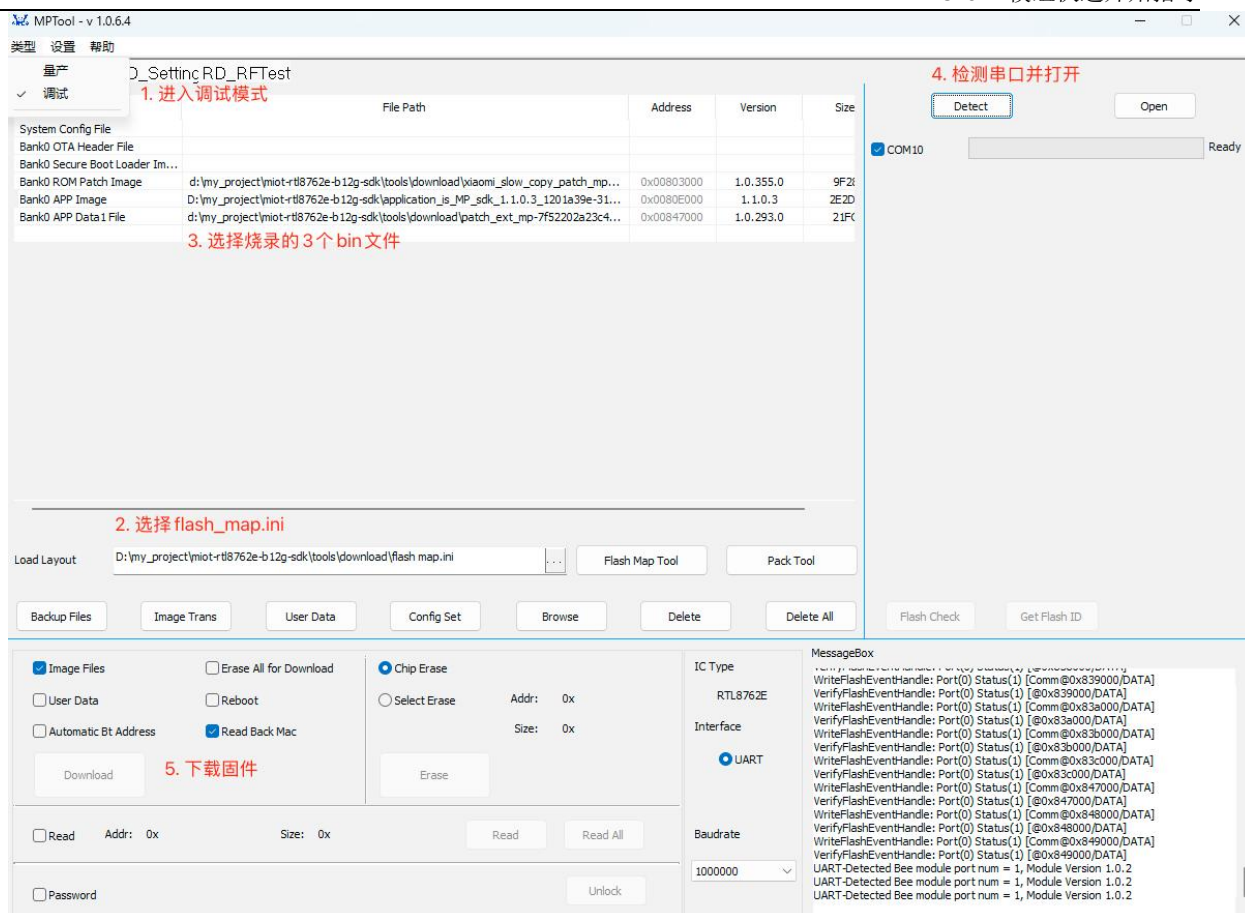
流过大导致的设备变砖，详见本文档第 12.3 章节。

6.2. 烧录步骤

1. 打开烧录工具，芯片类型选择 RTL8762E，然后类型选择调试；

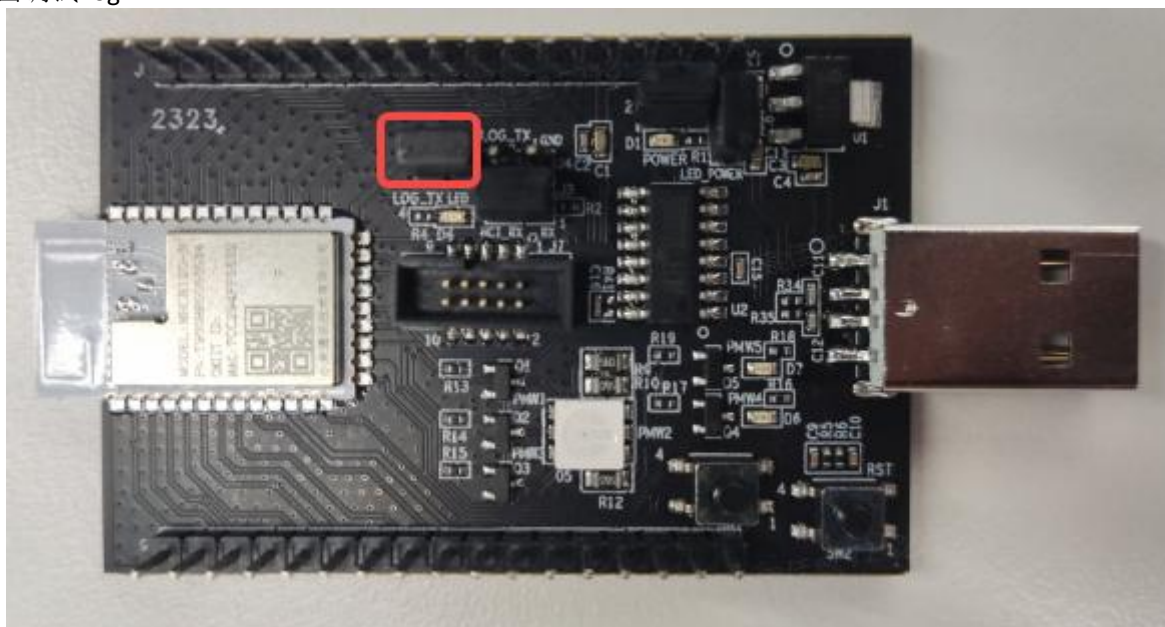


2. (1) 先选择 flash_map.ini，先选择烧录 bin 文件（其中 APP Image 使用编译生成的文件名带 MD5 值的 bin 文件，如：application_is_MP_sdk_1.1.0.3_1201a39e-fdb7091227e61a26294ff4e3b3f600db.bin），
(2) 点击 Detect 后显示开发板串口
(3) 点击 Open 后会对应串口后会提示 OK 表示打开成功
(4) 最后点击 Download 开始烧录，对应串口有进度条提示，进度条满且提 OK 则烧录成功。如若提示 Fail 则需复位下模组重新进行烧录，一次最多 8 个模组同时烧录。



7. 调试 log

默认使用 uart0 输出调试 log，将开发板 J4 处插针如下图短接，使用串口工具（波特率 115200）即可输出调试 log。



开启 LOG 输出后会增加约 20K 左右固件大小，可以通过关闭 CONFIG_MIBLE_SDK_LOG 编译选项关闭 lib 库日志输出。如需进一步关闭所有串口日志，可以按以下步骤操作：

1. # Add the following code (include '#')
2. # CONFIG_ARCH_LOWPUTC is not set
3. # CONFIG_DEV_CONSOLE is not set
- 4.
5. # Delete the following code
6. CONFIG_MIBLE_SDK_LOG=y

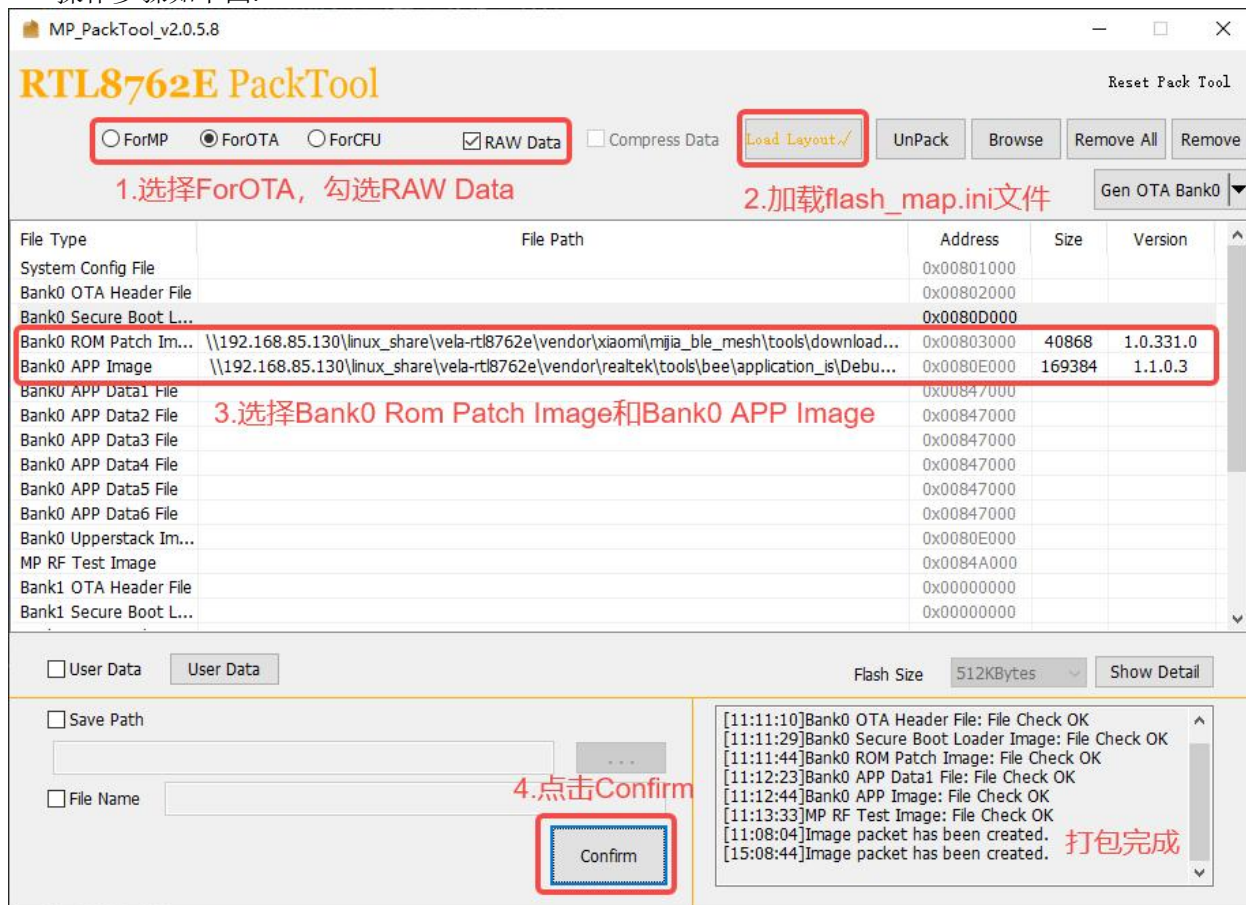
```

7. CONFIG_RTL876x_UART0=y
8. CONFIG_RTL876x_UART0_CONSOLE=y
9. CONFIG_RTL876x_UART0_RXBUFSIZE=256
10. CONFIG_RTL876x_UART0_TXBUFSIZE=256
11. CONFIG_RTL876x_UART=y
12. CONFIG_SYSLOG_DEVPATH="/dev/ttyS0"

```

8. 制作 OTA 包

- OTA 包制作使用 MP_PackTool 工具，工具位于代码目录 vendor/xiaomi/mijia_ble_mesh/tools，操作步骤如下图：

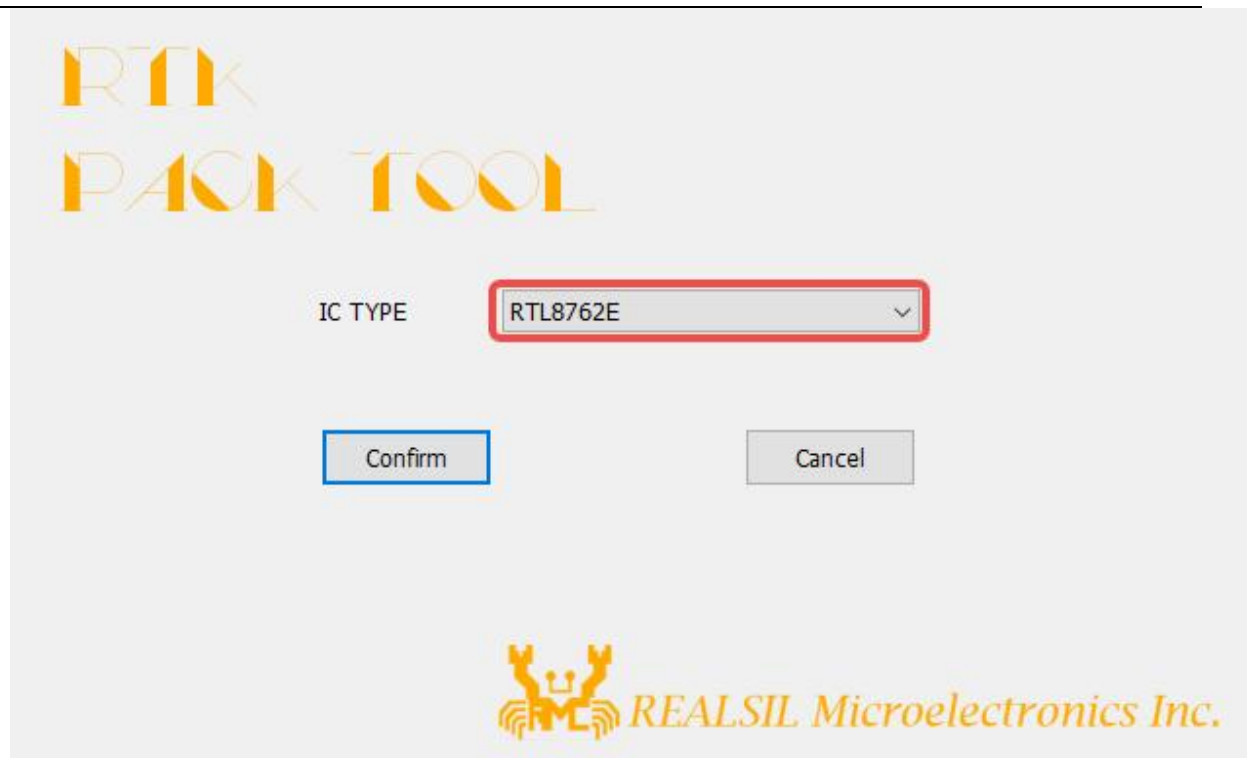


- 工具目录下（也可勾选 Save Path 选择保存路径）生成 ImgPacketFile_OTA_RAW 开头 bin 文件（如 ImgPacketFile_OTA_RAW-66c5d3651bda62ae91ac7b95df2484ed.bin）即生成 OTA 包，将该文件上传至小米 IoT 开发者平台。

***注意：**请勿将 Secure Boot 加入 ota 包，可忽略打包工具的 warning。若升级 boot 过程中断电，可能导致设备变砖。

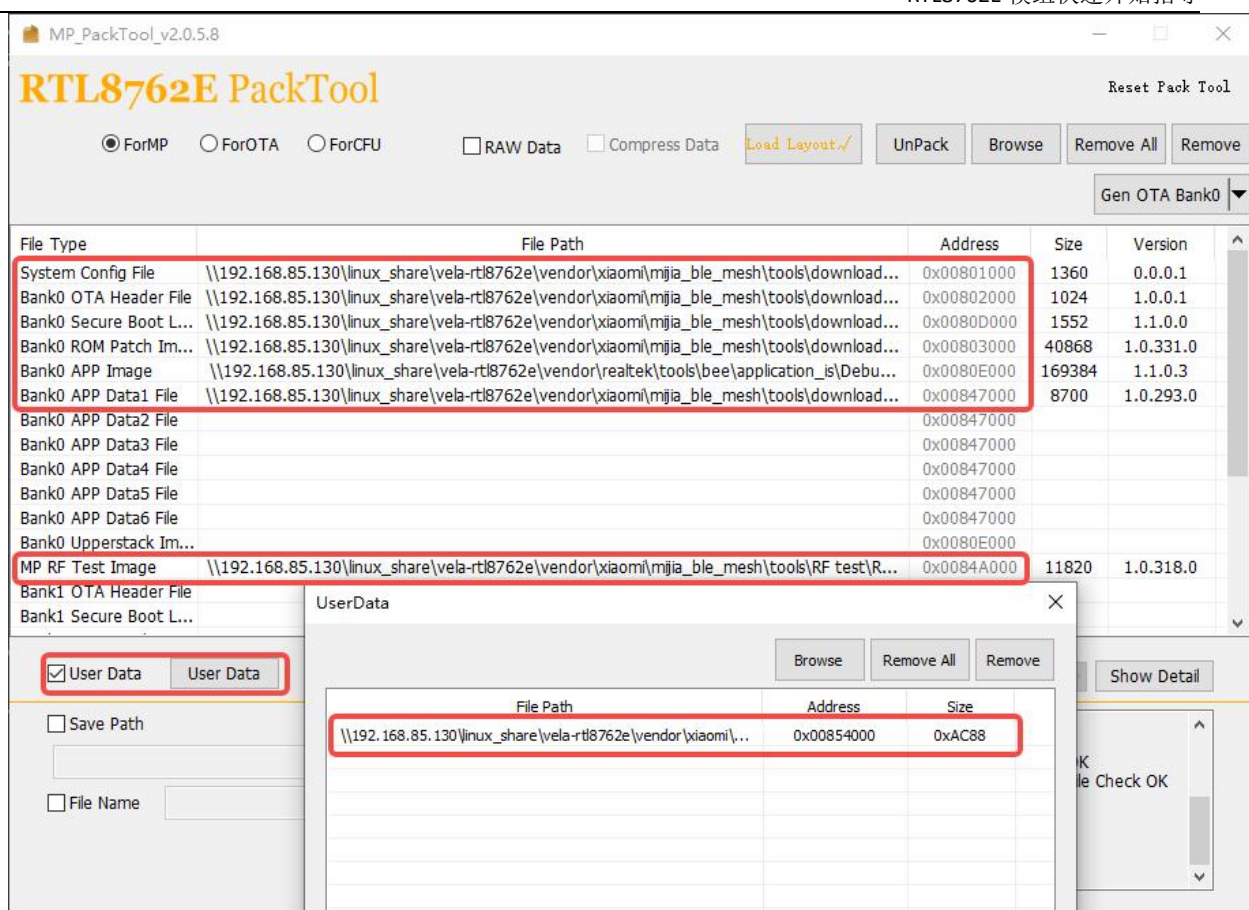
9. 制作厂包

- 厂包制作使用 MP_PackTool 工具，打开工具并选择 IC TYPE 为 RTL8762E；

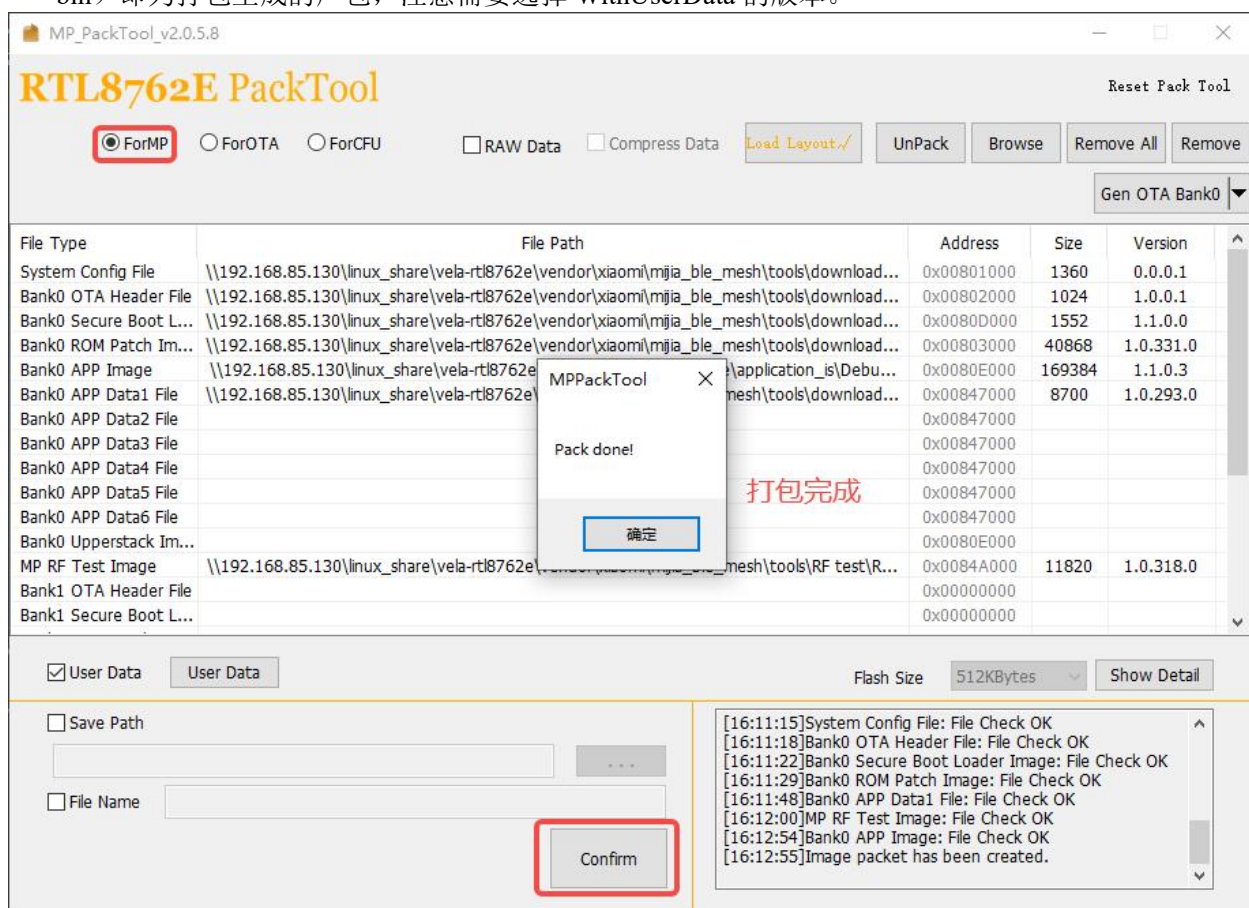


2. 按照如下图选择各个分区字段相应的 bin 文件，除了 Bank0 APP Image 是厂商自己编译的 APP 固件，其他文件位于代码目录 `vendor\xiaomi\mijia_ble_mesh\tools\download` 及 `vendor\xiaomi\mijia_ble_mesh\tools\factory_test`。

File Type	File	Address
System Config File	configFile_2023.09.18.10-2055ef9a6dfea005687062faa972cbbb.bin	0x801000
Bank0 OTA Header File	OTAHeader_Bank0_1.0.0.1-813f61fba2a9ef54bdffb8ef6d595640.bin	0x802000
Bank0 Secure Boot Loader Image	fsbl_mp_master##_1.1.0.0_dea0248d-818a246824b295b221dda5e3cc347abd.bin	0x80D000
Bank0 ROM Patch Image	XIAOMI_Patch_MP_master_1.0.352.0_10845ded-7788e7a5f1bba2442aa7251677ca1e27.bin	0x803000
Bank0 APP Image	application_is_MP_sdk_1.1.0.3_1201a39e-fdb7091227e61a26294ff4e3b3f600db.bin	0x80E000
Bank0 APP Date1 File	Patch_ext_MP-7f52202a23c41ea26bd2b257daec2925.bin	0x847000
MP RF Test Image	RF_TEST_MP_master_1.0.318.0_fee6cffd_0x0084A000-18f0fa0f88808f98eb9d8fbc5aa94401.bin	0x84A000
User Data	B12G_MAGICapp-2f89ef4350f696fed555993b35cf5906.bin 或 B12S_MAGICapp-0ed02c54dd64a28826b7b88d6313080b.bin (根据模组型号选择对应 bin 文件)	0x854000



3. 选择左上方 ForMP，点击下方 Confirm，工具目录下（也可勾选 Save Path 选择保存路径）生成 ImgPacketFile 开头 bin 文件（如 ImgPacketFile_Patch_1.0.352.0_App_1.1.0.3_WithUserData-33b644c29792db2d61b8730c6732a4e1.bin）即为打包生成的厂包，注意需要选择 WithUserData 的版本。



10. 分区表

分区号	分区名称	存储信息	起始地址	大小	备注
0	Reserved	芯片产测信息	0x800000	4KB	
1	OEM config	用户配置	0x801000	4KB	System Config File (MAC、频偏、发射功率、分区等)
2	OTA Header	OTA 配置信息	0x802000	4KB	Bank0 OTA Header File
3	Secure Boot	安全启动引导	0x80D000	4KB	Bank0 Secure Boot Loader Image
4	Patch	协议栈 Patch	0x803000	40KB	Bank0 ROM Patch Image
5	Application	用户主分区	0x80E000	228KB	Bank0 APP Image
6	App data	解压算法	0x847000	12KB	Bank0 APP Data1 File
7	OTA Patch	Patch OTA 分区	0x84A000	40KB	RF 测试 Patch
8	OTA Temp	APP OTA 分区	0x854000	160KB	压缩比 0.7，产测固件存放区
9	NVS	系统存储	0x87C000	12KB	
10	MIJIA CERT	证书链	0x87F000	4KB	

11. Mesh 色温灯工程

代码中编译./build.sh vendor/realtek/boards/rtl8762e/configs/app 配置的应用 Demo 中实现的是一个 mesh 色温灯示例，开发前先熟悉此示例代码；应用示例代码位于 vendor/xiaomi/mijia_ble_mesh/目录下，vendor/xiaomi/mijia_ble_mesh/src/main.c 中 main 函数是应用开发入口函数。

11.1. 宏定义配置

在 vendor/xiaomi/mijia_ble_mesh/main.c 中修改如下宏定义通过修改宏 PRODUCT_ID、MODEL_NAME、DEVELOPER_VERSION 值修改 PID、model name、版本号。

在 vendor/xiaomi/mijia_ble_mesh/model.c 中按照产品模板修改为对应宏定义。

```

1. #define PRODUCT_ID                8059
2. #define MODEL_NAME                "miot.light.tmp5"
3.
4. #define DEVELOPER_VERSION          127
5.
6. /*model.c model 模板选择*/
7. #define MI_MESH_TEMPLATE_LIGHTCTL

```

11.2. Mesh 状态回调

在 mi_config.cb.event.event 添加 Mesh 状态回调接口 user_event_cb，用户可在此接口中添加对应状态事件的处理，mesh 状态事件见下表：

Define	说明	操作
MIBLE_USER_PROV_CB_TYPE_UNPROV	设备未配网，在协议栈初始	设置 30min 广播超时时间，可

	化完成后进入	开启扫描以兼容遥控器功能
MIBLE_USER_PROV_CB_TYPE_PROVED	设备已配网，在协议栈初始化完成后进入	LED 调到默认亮度 30000，色温 4000，500ms 渐变
MIBLE_USER_PROV_CB_TYPE_RECOVERY	设备自恢复，在手动重置 20min 后，如果用户未在 APP 中删除，网关自动将设备恢复至重置前状态	LED 调到默认亮度 30000，色温 4000，500ms 渐变
MIBLE_USER_PROV_CB_TYPE_START	配网开始	
MIBLE_USER_PROV_CB_TYPE_COMPLETE	配网完成	
MIBLE_USER_PROV_CB_TYPE_FAIL	配网失败	
MIBLE_USER_PROV_CB_TYPE_RESET	配网解除	LED 呼吸 3 次之后重启
MIBLE_USER_LOGIN_CB_TYPE_START	登陆开始	
MIBLE_USER_LOGIN_CB_TYPE_COMPLETE	登陆完成	
MIBLE_USER_LOGIN_CB_TYPE_FAIL	登陆失败	
MIBLE_USER_GAP_CB_CONNECTED	建立 GATT 链接	启动定时器，20s 未登录主动断开连接
MIBLE_USER_GAP_CB_DISCONNECTED	断开 GATT 连接	
MIBLE_USER_SCENE_CB_ADD	场景添加回调	
MIBLE_USER_SCENE_CB_DELETE	场景删除回调	
MIBLE_USER_SCENE_CB_RECALL	场景调用回调	

示例代码：

```

1. static void user_event_cb(mible_user_event_type_t event, void *user_data)
2. {
3.     static const uint8_t *const strs[] = {
4.         [MIBLE_USER_PROV_CB_TYPE_UNPROV] = "Unprovision",
5.         [MIBLE_USER_PROV_CB_TYPE_PROVED] = "Provisioned",
6.         [MIBLE_USER_PROV_CB_TYPE_START] = "Provision start",
7.         [MIBLE_USER_PROV_CB_TYPE_COMPLETE] = "Provision completed",
8.         [MIBLE_USER_PROV_CB_TYPE_FAIL] = "Provision failed",
9.         [MIBLE_USER_PROV_CB_TYPE_RESET] = "Provision Reseted",
10.        [MIBLE_USER_PROV_CB_TYPE_RECOVERY] = "Provision Recoveried",
11.        [MIBLE_USER_LOGIN_CB_TYPE_START] = "Login start",
12.        [MIBLE_USER_LOGIN_CB_TYPE_COMPLETE] = "Login completed",
13.        [MIBLE_USER_LOGIN_CB_TYPE_FAIL] = "Login failed",
14.        [MIBLE_USER_GAP_CB_CONNECTED] = "Connected",
15.        [MIBLE_USER_GAP_CB_DISCONNECTED] = "Disconnected",
16.        [MIBLE_USER_SCENE_CB_ADD] = "Scene added",
17.        [MIBLE_USER_SCENE_CB_DELETE] = "Scene deleted",
18.        [MIBLE_USER_SCENE_CB_RECALL] = "Scene recalled",
19.    };
20.
21.    if (event >= ARRAY_SIZE(strs)) {
22.        return;
23.    }
24.
25.    printk("%s\n", strs[event]);
26.    switch (event)
27.    {
28.        case MIBLE_USER_PROV_CB_TYPE_UNPROV:
29.            printk("MIBLE_USER_PROV_CB_TYPE_UNPROV\n");
30.            is_provisioned = false;
31.            //mible_mesh_device_scan_set(MESH_SCAN_FULL);
32.            //mio_system_set_adv_timeout(1800000); // stop adv after 30min
33.            break;
34.        case MIBLE_USER_PROV_CB_TYPE_FAIL:
35.            is_provisioned = false;
36.            break;
37.        case MIBLE_USER_PROV_CB_TYPE_START:
38.            break;
39.        case MIBLE_USER_PROV_CB_TYPE_RECOVERY:
40.        case MIBLE_USER_PROV_CB_TYPE_COMPLETE:
41.    {

```

```

42.     is_provisioned = true;
43.     xiaomi_light.light_on = true;
44.     light_onoff_set(xiaomi_light.light_on);
45.     {
46.         int bright_warm = CT2WARM_BRIGHTNESS(xiaomi_light.lighttemp, xiaomi_light.l
ightness);
47.         int bright_cold = xiaomi_light.lightness_last - bright_warm;
48.         light_breath(&light_cw[0], 0, bright_cold, 500, 50, 3, true, NULL);
49.         light_breath(&light_cw[1], 0, bright_warm, 500, 50, 3, true, NULL);
50.     }
51. }break;
52. case MIBLE_USER_PROV_CB_TYPE_PROVED:
53.     if (reset_timer) {
54.         break;
55.     }
56.     is_provisioned = true;
57.     xiaomi_light.light_on = true;
58.     light_onoff_set(xiaomi_light.light_on);
59.     break;
60. case MIBLE_USER_PROV_CB_TYPE_RESET:{
61.     bool *reboot = user_data;
62.     *reboot = false;
63.     light_breath(&light_cw[0], 0, 30000, 1000, 50, 3, false, NULL);
64.     light_breath(&light_cw[1], 0, 30000, 1000, 50, 3, false, light_reset_change_don
e);
65. }break;
66. case MIBLE_USER_GAP_CB_CONNECTED:
67.     {
68.         int err;
69.
70.         err = mio_timer_create(&conn_timer,
71.                                mible_conn_timeout_handler,
72.                                MIBLE_TIMER_SINGLE_SHOT);
73.         if (err) {
74.             printk("Unable create conn timeout timer");
75.         } else {
76.             mio_timer_start(conn_timer, 20000, NULL);
77.         }
78.
79.         is_connected = true;
80.         is_logged = false;
81.     } break;
82. case MIBLE_USER_GAP_CB_DISCONNECTED:
83.     {
84.         if (conn_timer) {
85.             mio_timer_delete(conn_timer);
86.             conn_timer = NULL;
87.         }
88.
89.         is_connected = false;
90.     } break;
91. case MIBLE_USER_LOGIN_CB_TYPE_COMPLETE:
92.     {
93.         is_logged = true;
94.     } break;
95. default:
96.     break;
97. }
98. return;
99. }

```

11.3. OTA 状态回调

在 mi_config.cb.dfu_info.event 添加 OTA 状态回调接口 dfu_event_cb，用户可在此接口中添加对应状态事件的处理，OTA 状态事件见下表：

Define	说明
MIBLE_DFU_STATE_START	收到 OTA 指令，返回帧长度

MIBLE_DFU_STATE_UPGRADE_STATUS	新增网关升级功能，网关会询问设备是否可以升级，是否支持静默，设备需回复相应请求。
MIBLE_DFU_STATE_TRANSFER	开始传输固件，返回 ota buffer 中的断点序号
MIBLE_DFU_STATE_VERIFY	固件验签完成返回校验结果，可做固件版本判断
MIBLE_DFU_STATE_SWITCH	开始固件切换
MIBLE_DFU_STATE_PROCESS	每传输完成 2K 固件，返回当前帧序号
MIBLE_DFU_STATE_CANCEL	OTA 取消

示例代码：

```

1. static void dfu_event_cb(mible_dfu_state_t state, const mible_dfu_param_t *param)
2. {
3.     static const uint8_t *const strs[] = {
4.         [MIBLE_DFU_STATE_START] = "Start",
5.         [MIBLE_DFU_STATE_UPGRADE_STATUS] = "Upgrade Status",
6.         [MIBLE_DFU_STATE_TRANSFER] = "Transfer",
7.         [MIBLE_DFU_STATE_VERIFY] = "Verify",
8.         [MIBLE_DFU_STATE_SWITCH] = "Switch",
9.         [MIBLE_DFU_STATE_PROCESS] = "Process Report",
10.        [MIBLE_DFU_STATE_CANCEL] = "Cancel",
11.    };
12.
13.    mible_dfu_param_t *p_dfu = (mible_dfu_param_t *)param;
14.
15.    if (state >= ARRAY_SIZE(strs)) {
16.        return;
17.    } else if (state == MIBLE_DFU_STATE_UPGRADE_STATUS) {
18.        p_dfu->upgrade_status.rsp = p_dfu->upgrade_status.req;
19.        printk("Req silent %d, install %d, Rsp silent %d, install %d\n",
20.            p_dfu->upgrade_status._req.silent, p_dfu->upgrade_status._req.install,
21.            p_dfu->upgrade_status._rsp.silent, p_dfu->upgrade_status._rsp.install);
22.    } else if (state == MIBLE_DFU_STATE_VERIFY) {
23.        printk("verify result is %d, version %u.%u.%u_04u\n", p_dfu->verify.value,
24.            p_dfu->verify.version.major, p_dfu->verify.version.minor,
25.            p_dfu->verify.version.revision, p_dfu->verify.version.build);
26.        //TODO: p_dfu->verify.value return MIBLE_DFU_STATUS_ERR_INVALID_VERSION when ve
27.        rion is old
28.    }
29.    printk("DFU %s\n", strs[state]);
30. }

```

11.4. 设备信息回调

在 mi_config.cb.dev_info.req 添加设备信息回调接口 user_devinfo_callback，用户可在此接口中将设备信息通过回调在配网完成后上报至米家服务器，支持的设备信息类型见下表：

Define	说明
DEV_INFO_SUPPORT	支持的设备信息类型（必选）
DEV_INFO_MCU_VERSION	外部 MCU 版本号
DEV_INFO_DEVICE_SN	设备序列号（最大长度仅支持 18 字节，建议使用 DEV_INFO_NEW_SN）
DEV_INFO_HARDWARE_VERSION	设备硬件版本
DEV_INFO_LATITUDE	设备所在地经纬度
DEV_INFO_LONGITUDE	
DEV_INFO_VENDOR1	厂商自定义信息 1
DEV_INFO_VENDOR2	厂商自定义信息 2
DEV_INFO_NEW_SN	设备序列号（最大长度支持 62 字节）

示例代码：

```

1. static void user_devinfo_callback(mible_dev_info_type_t type, mible_dev_info_t *buf)
2. {
3.     static const uint8_t support_devinfo[] = {
4.         MIBLE_DEV_INFO_HARDWARE_VERSION,
5.         MIBLE_DEV_INFO_NEW_SN,

```



```

6.     };
7.
8.     switch (type) {
9.     case MIBLE_DEV_INFO_SUPPORT:
10.        buf->len = sizeof(support_devinfo);
11.        memcpy(buf->buff, support_devinfo, buf->len);
12.        break;
13.     case MIBLE_DEV_INFO_HARDWARE_VERSION:
14.        buf->len = strlen("MHCb12G");
15.        memcpy(buf->buff, "MHCb12G", buf->len);
16.        break;
17.     case MIBLE_DEV_INFO_NEW_SN:
18.        // maxlen 62
19.        buf->len = MIN(strlen("123456789/123456789/123456789/123456789/"
20.        "123456789/123456789/12"), 62);
21.        memcpy(buf->buff,
22.        "123456789/123456789/123456789/123456789/123456789/"
23.        "123456789/12",
24.        buf->len);
25.        break;
26.     default:
27.        buf->code = MI_ERR_NOT_FOUND;
28.        return;
29.     }
30.
31.     buf->code = MI_SUCCESS;
32.
33.     return;
34. }

```

11.5. Mesh 消息回调

Mesh 设备处于在线状态时，可以通过网关或蓝牙直接进行属性设置、读取以及方法执行，在 `mi_config.cb.spec.get` 添加属性读取回调接口 `do_spec_get`，`mi_config.cb.spec.set` 添加属性设置回调接口 `do_spec_set`，`mi_config.cb.spec.invoke` 添加方法执行回调接口 `do_spec_invoke`。

11.5.1. 读取属性回调 `do_spec_get`

入参 `bearer` 用于指定是 mesh 还是 gatt 承载控制，参数 `o` 用于指定操作属性 `ssid`、`piid` 及操作返回 `value`、状态码，函数接口原型如下：

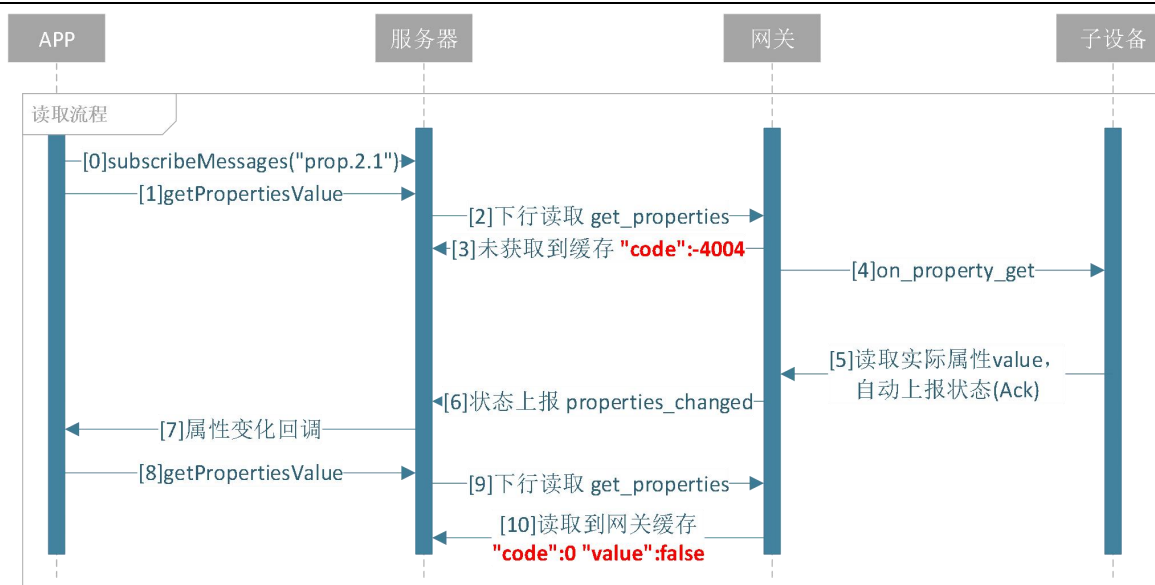
```

1. static void do_spec_get(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
2. {
3.     const struct mible_spec_piid_or_aaid_list *i;
4.
5.     if (bearer >= ARRAY_SIZE(spec_bearers)) {
6.         return;
7.     }
8.
9.     printk("%s %s ssid %d piid %d\n", __FUNCTION__, spec_bearers[bearer], o->ssid, o->p
iid);
10.
11.     i = find_spec_handler(o->ssid, o->piid);
12.     if (!i || !(i->get)) {
13.         return;
14.     }
15.
16.     i->get(bearer, o);
17. }

```

APP 读取属性状态流程如下：

- 判断服务 id 与属性 id 是否正确；
- 根据属性值的数据类型，使用 `mible_spec_property_value` 写入属性值（`*value`）；
- 操作成功，返回代码 `MIBLE_SPEC_OPERATION_OK`



11.5.2. 设置属性回调 do_spec_set

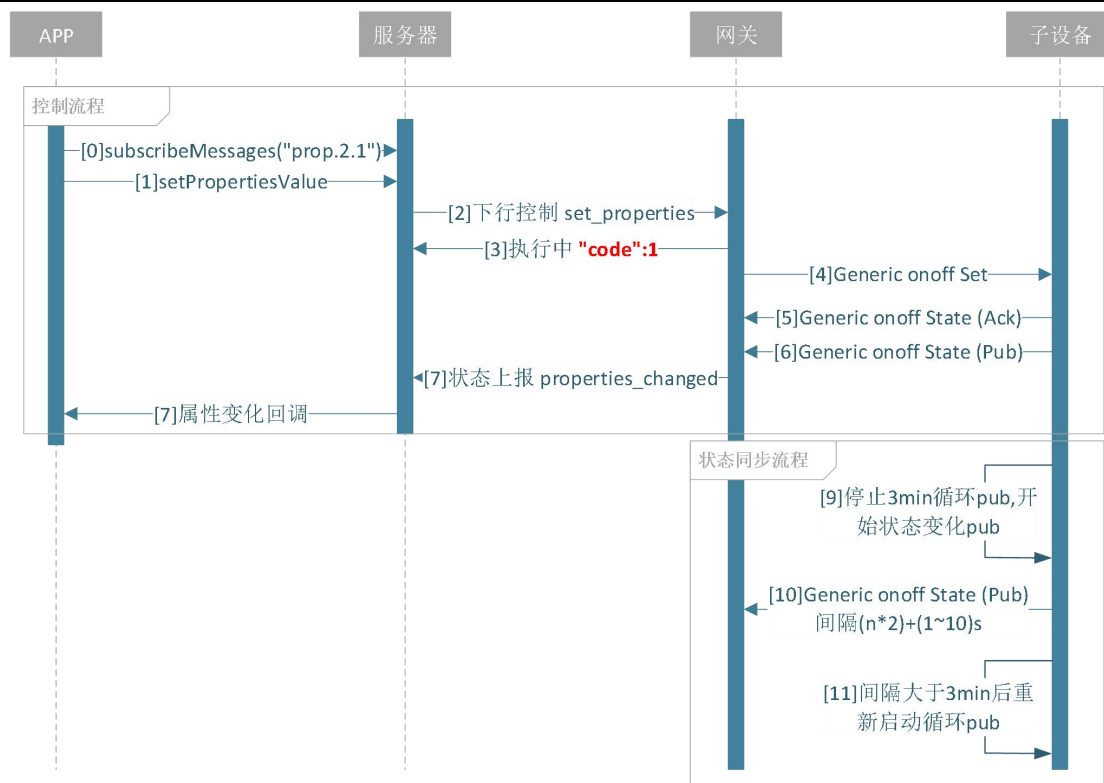
入参 bearer 用于指定是 mesh 还是 gatt 承载控制，参数 o 用于指定操作属性 ssid、piid、value 及操作返回状态码，函数接口原型如下：

```

1. static void do_spec_set(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
2. {
3.     const struct mible_spec_piid_or_aaid_list *i;
4.
5.     if (bearer >= ARRAY_SIZE(spec_bearers)) {
6.         return;
7.     }
8.
9.     printk("%s %s siid %d piid %d type %d val %08lx\n", __FUNCTION__, spec_bearers[bear
er],
10.           o->siid, o->piid, o->value.format, o->value.data.ulongValue);
11.
12.     i = find_spec_handler(o->siid, o->piid);
13.     if (!i || !(i->set)) {
14.         return;
15.     }
16.
17.     i->set(bearer, o);
18. }
  
```

APP 控制属性状态流程如下：

- 判断服务 id 与属性 id 是否正确；
- 判断数据格式是否正确，on 属性为 bool 类型；
- 判断数据取值是否合法，此处应与开放平台设置一致；
- 执行相应的操作，根据 on 属性开关 RGB 灯；
- 操作成功，返回代码 OPERATION_OK
- （可选）如果被设置的属性在 3min 循环上报表中，会在 3min 内补发 3~5 包 mesh 消息以保证状态及时同步。退避时间为 $n*2+rand(1,10)$ ，计算间隔大于 3min 时重启 3min 周期上报。



11.5.3. 执行方法回调 do_spec_invoke

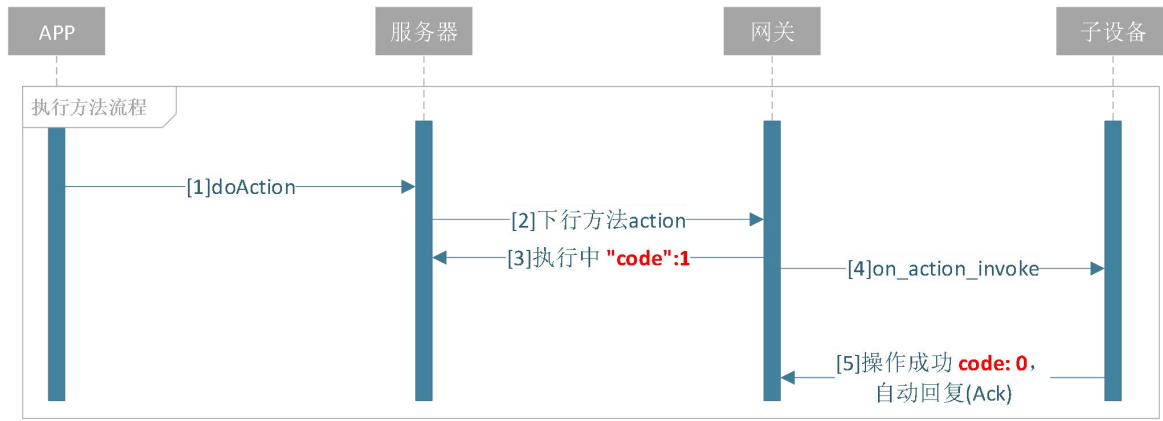
入参 `bearer` 用于指定是 `mesh` 还是 `gatt` 承载控制，参数 `o` 用于指定操作属性 `ssid`、`piid`、`value` 及操作状态码，函数接口原型如下：

```

1. static void do_spec_invoke(mible_spec_bearer_t bearer, mible_spec_action_operation_t *o)
2. {
3.     const struct mible_spec_piid_or_aaid_list *i;
4.
5.     if (bearer >= ARRAY_SIZE(spec_bearers)) {
6.         return;
7.     }
8.
9.     printk("%s %s ssid %d piid %d arg[size %d]\n", __FUNCTION__, spec_bearers[bearer],
10.        o->ssid, o->aaid,
11.        o->vals.size);
12.     i = find_spec_handler(o->ssid, o->aaid);
13.     if (!i || !(i->invoke)) {
14.         return;
15.     }
16.
17.     i->invoke(bearer, o);
18. }
  
```

APP 执行方法流程如下：

- 判断服务 id 与方法 id 是否正确；
- 检查输入参数个数，将输出参数个数设置为 0；
- 执行灯状态反转操作；
- 上报灯状态属性变化。



11.5.4. 属性操作接口定义

用户需要按照对应回调接口原型定义具体属性操作接口，并将定义属性接口添加至 `spec_handlers` 定义中，例如色温灯开关属性定义 `P_2_1_On_doGet`、`P_2_1_On_doSet`、`P_2_1_On_doAction` 三个属性操作接口，并在 `spec_handlers` 定义服务 `SIID=2` 的 `MIBLE_SPEC_SIID_START(2)` 下添加 `MIBLE_SPEC_HANDLER(1, P_2_1_On_doGet, P_2_1_On_doSet, P_2_1_On_doAction)`，其中第一个参数 1 表示 `PIID=1`，`MIBLE_SPEC_SIID_START(2)` 中的 2 表示 `SIID` 为 2，可参考示例代码新增对应属性操作接口；

示例代码如下：

```

1. static void P_1_1_Manufacturer_doGet(mible_spec_bearer_t bearer, mible_spec_property_op
   operation_t *o)
2. {
3. }
4.
5. static void P_1_2_Model_doGet(mible_spec_bearer_t bearer, mible_spec_property_operation
   _t *o)
6. {
7. }
8.
9. static void P_1_3_SerialNumber_doGet(mible_spec_bearer_t bearer, mible_spec_property_op
   eration_t *o)
10. {
11. }
12.
13. static void P_1_4_FirmwareRevision_doGet(mible_spec_bearer_t bearer,
14.         mible_spec_property_operation_t *o)
15. {
16. }
17.
18. static void P_2_1_On_doGet(mible_spec_bearer_t bearer, mible_spec_property_operation_t
   *o)
19. {
20.     mible_spec_property_value_new_boolean(&o->value, xiaomi_light.light_on);
21. }
22.
23. static void P_2_2_Brightness_doGet(mible_spec_bearer_t bearer, mible_spec_property_oper
   ation_t *o)
24. {
25.     mible_spec_property_value_new_ushort(&o->value, xiaomi_light.lightness);
26. }
27.
28. static void P_2_3_ColorTemperature_doGet(mible_spec_bearer_t bearer,
29.         mible_spec_property_operation_t *o)
30. {
31.     mible_spec_property_value_new_ulong(&o->value, xiaomi_light.lighttemp);
32. }
33.
34. static void P_2_1_On_doSet(mible_spec_bearer_t bearer, mible_spec_property_operation_t
   *o)
35. {
36.     if (o->value.format != MIBLE_SPEC_PROPERTY_FORMAT_BOOL)
37.     {
38.         o->code = MIBLE_SPEC_OPERATION_ERROR_VALUE;
39.         return;

```

```

40.     }
41.
42.     light_onoff_set(o->value.data.boolean);
43.
44.     o->code = MIBLE_SPEC_OPERATION_OK;
45. }
46.
47. static void P_2_2_Brightness_doSet(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
48. {
49.     if (o->value.format != MIBLE_SPEC_PROPERTY_FORMAT_USHORT && o->value.format != MIBLE_SPEC_PROPERTY_FORMAT_NUMBER)
50.     {
51.         o->code = MIBLE_SPEC_OPERATION_ERROR_VALUE;
52.         return;
53.     }
54.
55.     light_brightness_set(o->value.data.ushortValue);
56.
57.     o->code = MIBLE_SPEC_OPERATION_OK;
58. }
59.
60. static void P_2_3_ColorTemperature_doSet(mible_spec_bearer_t bearer,
61.                                           mible_spec_property_operation_t *o)
62. {
63.     if (o->value.format != MIBLE_SPEC_PROPERTY_FORMAT_ULONG && o->value.format != MIBLE_SPEC_PROPERTY_FORMAT_NUMBER)
64.     {
65.         o->code = MIBLE_SPEC_OPERATION_ERROR_VALUE;
66.         return;
67.     }
68.
69.     light_temperature_set(o->value.data.ulongValue);
70.
71.     o->code = MIBLE_SPEC_OPERATION_OK;
72. }
73.
74. static void P_2_1_On_doAction(mible_spec_bearer_t bearer, mible_spec_action_operation_t *o)
75. {
76.     mible_spec_property_value_t value;
77.
78.     if(xiaomi_light.light_on){
79.         xiaomi_light.light_on = false;
80.     }else{
81.         xiaomi_light.light_on = true;
82.     }
83.     light_onoff_set(xiaomi_light.light_on);
84.
85.     mible_spec_property_value_new_boolean(&value, xiaomi_light.light_on);
86.     mio_mesh_properties_changed(2, 1, &value);
87.
88.     o->vals.size = 0;
89.     o->code = MIBLE_SPEC_OPERATION_OK;
90. }
91.
92. static void P_128_1_GMT_Offset_doSet(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
93. {
94. }
95.
96. static void P_128_2_Weather_doSet(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
97. {
98. }
99.
100. static void P_128_3_UTC_Time_doSet(mible_spec_bearer_t bearer, mible_spec_property_operation_t *o)
101. {
102. }

```

```

103.
104. MIBLE_SPEC_DEFINE(spec_handlers)
105.     MIBLE_SPEC_SIID_START(1)
106.         MIBLE_SPEC_HANDLER(1, P_1_1_Manufacturer_doGet, NULL, NULL),
107.         MIBLE_SPEC_HANDLER(2, P_1_2_Model_doGet, NULL, NULL),
108.         MIBLE_SPEC_HANDLER(3, P_1_3_SerialNumber_doGet, NULL, NULL),
109.         MIBLE_SPEC_HANDLER(4, P_1_4_FirmwareRevision_doGet, NULL, NULL),
110.     MIBLE_SPEC_SIID_END
111.
112.     MIBLE_SPEC_SIID_START(2)
113.         MIBLE_SPEC_HANDLER(1, P_2_1_On_doGet, P_2_1_On_doSet, P_2_1_On_doAction),
114.         MIBLE_SPEC_HANDLER(2, P_2_2_Brightness_doGet, P_2_2_Brightness_doSet, NULL),
115.         MIBLE_SPEC_HANDLER(3, P_2_3_ColorTemperature_doGet, P_2_3_ColorTemperature_doS
116. et, NULL),
117.     MIBLE_SPEC_SIID_END
118.
119.     MIBLE_SPEC_SIID_START(128)
120.         MIBLE_SPEC_HANDLER(1, NULL, P_128_1_GMT_Offset_doSet, NULL),
121.         MIBLE_SPEC_HANDLER(2, NULL, P_128_2_Weather_doSet, NULL),
122.         MIBLE_SPEC_HANDLER(3, NULL, P_128_3_UTC_Time_doSet, NULL),
123.     MIBLE_SPEC_SIID_END
124. MIBLE_SPEC_END

```

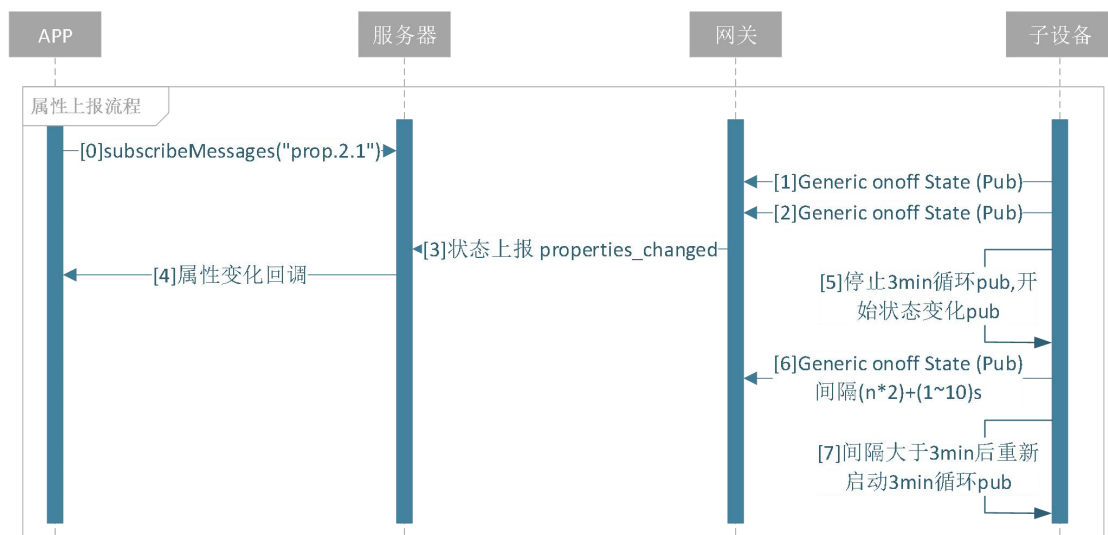
11.6. Mesh 属性上报

11.6.1. 属性上报 `miio_mesh_properties_changed`

使用 `miio_mesh_properties_changed` 函数接口主动上报属性状态值，该接口需要传入属性的服务 id (siid)，属性 id (piid) 与属性值 (*value)。

属性上报流程如下：

- 根据属性值的数据类型，使用 `mible_spec_property_value_new` 写入属性值 (*value)；
- 填入 siid 和 piid 调用发送函数。
- (可选) 如果上报的属性在 3min 循环上报表中，会自动在 3min 内补发 3~5 包 mesh 消息以保证状态及时同步。退避时间为 $n*2+rand(1,10)$ ，计算间隔大于 3min 时重启 3min 周期上报。



示例代码：

```

1. static void P_2_1_On_doChange(bool onoff)
2. {
3.     mible_spec_property_value_t value;
4.     mible_spec_property_value_new_boolean(&value, onoff);
5.     miio_mesh_properties_changed(2, 1, &value);
6. }

```

11.6.2. 属性周期上报 `miio_mesh_properties_period_publish_init`

使用 `miio_mesh_properties_period_publish_init` 函数接口可设置属性周期上报，该接口需要传入属

性的服务 id (siid)，属性 id (piid)与上报周期(period)。米家 SDK 会根据设置的上报周期(period) 将属性添加到 3min 循环上报表或用户周期上报表中，到达上报周期后，会通过读取属性回调函数 on_property_get 获取最新属性值上报至网关。

示例代码：

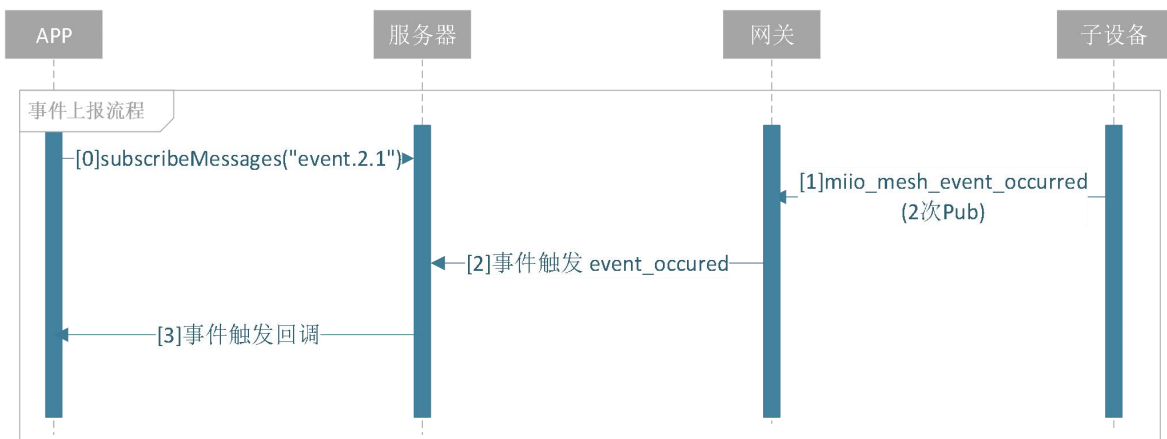
```
1. static void user_event_cb(mible_user_event_type_t event, void *user_data)
2. {
3.
4.     switch (event)
5.     {
6.         case MIBLE_USER_PROV_CB_TYPE_PROVED:
7.             // add Left/Right onoff to 3min circle publish list
8.             mio_mesh_properties_period_publish_init(2, 1, 0);
9.             mio_mesh_properties_period_publish_init(3, 1, 0);
10.            // add Temperature and Humidity to publish list, period 30min
11.            mio_mesh_properties_period_publish_init(5, 1, 1800);
12.            mio_mesh_properties_period_publish_init(5, 2, 1800);
13.            break;
14.        }
15.    }
```

11.7. Mesh 事件上报

11.7.1. 事件触发上报 mio_mesh_event_occurred

使用 mio_mesh_event_occurred 函数接口上报事件，该接口需要传入事件的服务 id (siid)，事件 id (eiid)与事件参数(*args)。

事件上报流程如下：



示例代码：

```
1. void E_6_1_Click_doEvent(void)
2. {
3.     if(mio_mesh_event_occurred(6, 1, NULL) != 0){
4.         printk("send_event_occurred failed!\n");
5.     }
6. }
7.
8. void E_6_2_Long_Press_doEvent(uint32_t arg1, int64_t arg2)
9. {
10.     mible_spec_arguments_t newArgs;
11.     mible_spec_argument_t arguments[2];
12.     newArgs->arguments[0].piid = 1;
13.     mible_spec_property_value_new_long(&newArgs->arguments[0].value, arg1);
14.     newArgs->arguments[1].piid = 2;
15.     mible_spec_property_value_new_longlong(&newArgs->arguments[1].value, arg2);
16.
17.     if(mio_mesh_event_occurred(6, 2, &newArgs) != 0){
18.         printk("send_event_occurred failed!\n");
19.     }
20. }
```


11.8. 服务器信息请求

目前仅开放 MIOT 平台通用属性的上行请求，可通过该接口判断设备是否已连接上网关，其 siid 与 piid 定义如下：

服务 siid	属性 piid	功能说明	备注
128	1	设备时区	根据 app 或网关的地理信息计算时区
128	2	地区天气码	根据设备或网关的经纬度拉取本地天气
128	3	Utc 时间戳	自 1970 年开始的 utc 秒数

11.8.1. 请求服务器信息 int miiio_mesh_request_property (uint8_t type)

以每小时获取一次服务器信息为例：

- 初始化后启动 poll_second 定时器，每小时调用 miiio_mesh_request_property 函数接口拉取服务器信息；
 - 服务器返回信息使用写属性方式在 on_property_set 回调函数中，更新对应的属性值；
- 示例代码：

```

1. static void P_128_1_GMT_Offset_doSet(mible_spec_bearer_t bearer, mible_spec_property_op
   operation_t *o)
2. {
3. }
4.
5. static void P_128_2_Weather_doSet(mible_spec_bearer_t bearer, mible_spec_property_opera
   tion_t *o)
6. {
7. }
8.
9. static void P_128_3_UTC_Time_doSet(mible_spec_bearer_t bearer, mible_spec_property_oper
   ation_t *o)
10. {
11. }
12.
13. #define SERVER_GMT_OFFSET 1
14. #define SERVER_WEATHER 2
15. #define SERVER_UTC_TIME 3
16.
17. void P_128_1_GMT_Offset_doRequest(void)
18. {
19.     if(miiio_mesh_request_property(SERVER_GMT_OFFSET) != 0){
20.         MI_LOG_ERROR("send_property_request failed!\n");
21.     }
22. }
23.
24. void P_128_2_Weather_doRequest(void)
25. {
26.     if(miiio_mesh_request_property(SERVER_WEATHER) != 0){
27.         MI_LOG_ERROR("send_property_request failed!\n");
28.     }
29. }
30.
31. void P_128_3_UTC_Time_doRequest(void)
32. {
33.     if(miiio_mesh_request_property(SERVER_UTC_TIME) != 0){
34.         MI_LOG_ERROR("send_property_request failed!\n");
35.     }
36. }

```

11.9. Mesh2.0 功能

11.9.1. 秒级配网 (Beta)

该功能需要搭配中枢网关使用，需要打开 CONFIG_BT_MESH_PSK 宏并注册初始化函数。开启该功能后，未配网模式下会自动开启 100%扫描，该功能不建议低功耗设备使用。

11.9.2. 远程 OTA

该功能需要搭配中枢网关使用，需要打开 CONFIG_BT_MESH_PATH 宏并注册初始化函数。开发者

需要在 IOT 平台中的“固件开发”页面修改自动升级配置（该配置项需要联系产品经理在后台打开）。

支持静默传输：开启后将在凌晨 2:00-6:00 进行自动静默传输，传输过程中要求设备不可发出声音和灯光效果。网关会在 OTA 启动时通过 MIBLE_DFU_STATE_UPGRADE_STATUS 回调询问设备当前状态是否支持静默传输，设备回复_rsp.silent=1 后开启传输流程。

支持自动升级：开启后将在静默传输完成后自动切换固件，固件切换过程中要求设备不可发出声音和灯光效果。网关会在 OTA 启动时通过 MIBLE_DFU_STATE_UPGRADE_STATUS 回调询问设备当前状态是否支持静默安装，设备回复_rsp.silent=1 和_rsp.install=1 后会自动安装固件。

如果设备不支持静默安装，则传输完成后 APP 中会提示用户升级。芯片 BOOT 阶段无法使用 PWM 且无法操作 GPIO，在开灯状态下应拒绝自动安装固件。

示例代码：

```
1. static void dfu_event_cb(mible_dfu_state_t state, const mible_dfu_param_t *param)
2. {
3.     static const uint8_t *const strs[] = {
4.         [MIBLE_DFU_STATE_START] = "Start",
5.         [MIBLE_DFU_STATE_UPGRADE_STATUS] = "Upgrade Status",
6.         [MIBLE_DFU_STATE_TRANSFER] = "Transfer",
7.         [MIBLE_DFU_STATE_VERIFY] = "Verify",
8.         [MIBLE_DFU_STATE_SWITCH] = "Switch",
9.         [MIBLE_DFU_STATE_PROCESS] = "Process Report",
10.        [MIBLE_DFU_STATE_CANCEL] = "Cancel",
11.    };
12.
13.    mible_dfu_param_t *p_dfu = (mible_dfu_param_t *)param;
14.
15.    if (state >= ARRAY_SIZE(strs)) {
16.        return;
17.    } else if (state == MIBLE_DFU_STATE_UPGRADE_STATUS) {
18.        p_dfu->upgrade_status.rsp = p_dfu->upgrade_status.req;
19.        printk("Req silent %d, install %d, Rsp silent %d, install %d\n",
20.            p_dfu->upgrade_status.req.silent, p_dfu->upgrade_status.req.install,
21.            p_dfu->upgrade_status.rsp.silent, p_dfu->upgrade_status.rsp.install);
22.    }
23.
24.    printk("DFU %s\n", strs[state]);
25. }
```

11.9.3. 一致性场景 (Beta)

该功能需要搭配中枢网关使用。Mesh 2.0 SDK 支持一致性场景方案，提升开关灯等场景下的多设备同步体验。每个设备最多支持存储 32 个场景，每个场景最多支持 10 个属性，并支持自定义延时。

场景添加/删除/调用时通过 MIBLE_USER_SCENE_CB_XXX 回调到应用层，相关逻辑由 SDK 处理，用户可无需关注回调内容。

在场景调用后，SDK 会在 **delay 时间后依次 set 场景内的各个属性**。开发者需保证设备正常响应，例如关灯时 set 亮度/色温，响应值变化但灯不被打开；关灯时 set 开关/亮度/色温，可正常调节到目标值，灯光渐变自然调光无卡顿。

11.10. 私有广播发送

SDK 预留了一个广播集接口 mible_gap_user_noconn_adv_start，广播间隔需设置为不小于 20ms，发包数量、广播内容可自行设置。

示例代码：

```
1. static const struct bt_data ad[] = {
2.     BT_DATA_BYTES(BT_DATA_FLAGS, BT_LE_AD_NO_BREDR),
3.     BT_DATA_BYTES(BT_DATA_UUID16_ALL, 0xaa, 0xfe),
4.     BT_DATA_BYTES(BT_DATA_SVC_DATA16,
5.         0xaa, 0xfe, /* Eddystone UUID */
6.         0x10, /* Eddystone-URL frame type */
7.         0x00, /* Calibrated Tx power at 0m */
8.         0x00, /* URL Scheme Prefix http://www. */
9.         'z', 'e', 'p', 'h', 'y', 'r',
10.        'p', 'r', 'o', 'j', 'e', 'c', 't',
```

```

11.         0x08) /* .org */
12. };
13.
14. static void button_process_handler(void *args)
15. {
16.     printk("button pressed!\n");
17.
18.     mible_gap_user_noconn_adv_start(20, 8, ad, ARRAY_SIZE(ad));
19. }

```

11.11. 设备重置

设备已配网并连入网关状态下通过 APP 删除设备，Mesh 网关会发送 Node Reset 命令给设备，SDK 收到指令后会先清除配网信息后发送 MIBLE_USER_PROV_CB_TYPE_RESET 回调给应用层，在回调中可以清除用户信息。

此外，设备必须支持手动重置，例如灯品类通过上下电 5 次(8s 内)重置设备，开关品类通过长按按键（长按 5s 松开后再长按 5s）重置设备。主动重置需调用 miio_system_restore 接口清除配网信息。

示例代码：

```

1. static bool light_flash_restore(void)
2. {
3.     uint32_t pwr_cnt = 0;
4.     int ret;
5.
6.     ret = miio_record_read(RECORD_ID_POWER_ON_COUNT,
7.                            (uint8_t *)&pwr_cnt, sizeof(pwr_cnt));
8.     if (ret != MI_SUCCESS) {
9.         goto store;
10.    }
11.
12.    MI_LOG_INFO("Power on count %d\n", pwr_cnt);
13.
14.    if (pwr_cnt >= SWITCH_POWER_ON_COUNT) {
15.        return true;
16.    }
17.
18. store:
19.     pwr_cnt++;
20.
21.     ret = miio_timer_create(&pwn_timer,
22.                             pwr_timer_handler,
23.                             MIBLE_TIMER_SINGLE_SHOT);
24.     if (ret) {
25.         MI_LOG_ERROR("Unable to create timer for power reset");
26.         return false;
27.     }
28.
29.     miio_timer_start(pwn_timer, 1000, (void *)pwr_cnt);
30.
31.     return false;
32. }
33.
34. static void node_restore_timer_handler(void * arg)
35. {
36.     int err;
37.
38.     err = miio_record_delete(RECORD_ID_POWER_ON_COUNT);
39.     if (err) {
40.         MI_LOG_ERROR("Unable to delete recoid id:0x%04x",
41.                      RECORD_ID_POWER_ON_COUNT);
42.     }
43.
44.     miio_system_restore();
45. }
46.
47. void main(void)
48. {

```

```

49.     int err;
50.
51.     if (light_flash_restore()) {
52.         MI_LOG_INFO("mio_system_restore\n");
53.         /* Call restore later, since mijia uninit yet. */
54.         err = mible_timer_create(&reset_timer,
55.                                 node_restore_timer_handler,
56.                                 MIBLE_TIMER_SINGLE_SHOT);
57.         if (err) {
58.             MI_LOG_ERROR("Unable to create restore timer");
59.             return;
60.         }
61.
62.         /* Indicate system restore in progress. */
63.         led_set(led_red_fd, 65535);
64.         led_set(led_gree_fd, 65535);
65.
66.         mible_timer_start(reset_timer, 1000, NULL);
67.     }
68. }

```

12. 单火开关工程

代码中编译./build.sh vendor/realtek/boards/rtl8762e/configs/app_pm 配置，此配置默认打开低功耗功能，其他应功能开发同 Mesh 色温灯工程。

12.1. 宏定义配置

在 vendor/xiaomi/mijia_ble_mesh/main_lpn.c 中修改如下宏定义通过修改宏 PRODUCT_ID 、MODEL_NAME、DEVELOPER_VERSION 值修改 PID、model name、版本号。

在 vendor/xiaomi/mijia_ble_mesh/model.c 中按照产品模板修改为对应宏定义。

```

8. #define PRODUCT_ID                19274
9. #define MODEL_NAME                " xiaomi.switch.mesh2"
10.
11. #define DEVELOPER_VERSION         138
12.
13. /*model.c model 模板选择*/
14. #define MI_MESH_TEMPLATE_TWO_KEY_SWITCH

```

12.2. 上电功耗

单火开关开态取电能力较弱，接小功率灯时可能因电压不足造成芯片无法启动。开发者设计软件时，可在 main 函数 io 初始化后适当增加延时，此时芯片进入 dlps 模式供电容充电，之后再操作继电器。

SDK 的 boot 时间约为 370ms，平均电流 4.5mA。demo 中增加了 2s 上电延时，开发者可根据实际灯具兼容性优化启动时间。

示例代码：

```

1. int main(int argc, FAR char *argv[])
2. {
3.     int err;
4.
5.     printk("Model Name %s PID %d Develop Version %d\n",
6.           mi_config.cfg.model_name, mi_config.cfg.pid, mi_config.cfg.dev_version);
7.
8.     err = wdg_init();
9.     if (err) {
10.         printk("watchdog init failed (err %d)\n", err);
11.         return err;
12.     }
13.
14.     err = board_init();
15.     if (err) {
16.         printk("board init failed (err %d)\n", err);
17.         return err;

```

```

18.     }
19.
20.     /* let's wait for a while avoid power reset */
21.     // Note: delay init bt for Capacitor charging when Relay on
22.     // Boot time ~ 0.37s * 4.5mA, delay time ~ 2s * 0.02mA
23.     if (IS_ENABLED(CONFIG_PM)) {
24.         k_sleep(K_SECONDS(2));
25.     }
26.     //.....
27. }

```

12.3. 固件搬运功耗

单火开关开态取电能力较弱，OTA 固件传输完成后，建议关闭继电器使用关态取电电路以提高取电能力。固件搬运时间~10s，平均电流~5mA，若电源供电不足可能会导致无法完成固件搬运导致的“假变砖”现象。

SDK 为单火开关应用提供了降速搬运 Patch，默认位置 vendor\xiaomi\mijia_ble_mesh\tools\download\XIAOMI_SLOW_COPY_Patch_MP_master_1.0.352.0_10845ded-eecc17161ace68bee5f8d2c9460fa3a2.bin。

降速搬运 Patch 会在 0x87B000 区域保存上电次数，正常搬运电流为 10s*5mA，若搬运期间发生掉电现象，会切换至 30s*2mA 速度搬运。使用该 Patch 时需注意 OTA **签名固件大小不得超过 196KB**。

12.4. 扫描窗口

单火开关使用“硬分时扫描”方式接收控制消息，开发者可使用 `miao_mesh_set_scan_level` 接口设置扫描等级。Demo 中将正常模式下的扫描等级设置为 **NORMAL**，即 20%扫描窗口，此时的待机电流~1.3mA；如果开启防闪烁模式或设备与手机直连，扫描等级设置为 **LOW**，即 15%扫描窗口，此时的待机电流~1mA；开发者如果需要在未配网模式下扫描广播，可以将扫描窗口设置为 **LOW**，否则设置为 **OFF**。

示例代码：

```

1. void user_scan_check(void)
2. {
3.     if(is_provisioned){
4.         if(two_key_switch.anti_flk || is_connected){
5.             // if flk mode or connected, set scan low mode(15%)
6.             miao_mesh_set_scan_level(MESH_SCAN_LOW);
7.         }else{
8.             // set scan normal mode(20%)
9.             miao_mesh_set_scan_level(MESH_SCAN_NORMAL);
10.        }
11.    }else{
12.        // if need scan when unprov, set scan low mode(15%)
13.        miao_mesh_set_scan_level(MESH_SCAN_NORMAL);
14.    }
15. }
16.
17. static void user_event_cb(mible_user_event_type_t event, void *user_data)
18. {
19.     switch (event)
20.     {
21.     case MIBLE_USER_PROV_CB_TYPE_UNPROV:
22.         printk("MIBLE_USER_PROV_CB_TYPE_UNPROV\n");
23.         is_provisioned = false;
24.         user_scan_check();
25.         break;
26.     case MIBLE_USER_PROV_CB_TYPE_RECOVERY:
27.     case MIBLE_USER_PROV_CB_TYPE_COMPLETE:
28.     case MIBLE_USER_PROV_CB_TYPE_PROVED:
29.         printk("MIBLE_USER_PROV_CB_TYPE_PROVED\n");
30.         is_provisioned = true;
31.         user_scan_check();
32.         break;
33.     case MIBLE_USER_GAP_CB_CONNECTED:
34.         is_connected = true;
35.         user_scan_check();
36.         break;
37.     case MIBLE_USER_GAP_CB_DISCONNECTED:

```

```

38.         is_connected = false;
39.         user_scan_check();
40.     break;
41. }
42. }

```

12.5. DLPS 回调

Vela OS 处于 Idle task 且蓝牙和外设满足进入 DLPS 条件时，在 `gpio_check_dlps` 回调中返回 `true`，系统会自动进入 DLPS 模式以降低功耗(~3uA)。

在 DLPS 下，clock, cpu, Peripherals 等模块会掉电，掉电前必须在 `gpio_enter_dlps` 回调中保存必要的数据以恢复系统。

退出 DLPS 模式时，需要在 `gpio_exit_dlps` 回调中将各模块重新上电并恢复到之前的状态。

在 DLPS 下，Pad 信号可以将系统从 DLPS 状态唤醒，可以在 `gpio_wakeup_dlps` 回调判断唤醒源，并暂时关闭 DLPS 以响应中断。

Demo 中配置了按键 P4_2 的中断和 Pad 唤醒，并在中断和进入 DLPS 前更改 IO 极性以实现双边沿触发。其他外设可以参考 8762C 芯片进出 DLPS 的代码，进行寄存器的保存与恢复。

示例代码：

```

1. #define BUTTON_0          P4_2
2. #define GPIO_PIN_BUTTON_0  GPIO_GetPin(BUTTON_0)
3. #define GPIO_BUTTON0_IRQn  GPIO30_IRQn
4.
5. #define DATA_RAM_FUNCTION __attribute__((section(".app.data_ram.text")))
6. static uint32_t gpio_storereg[10];
7. bool IO_DLPS_Enter_Allowed = true;
8.
9. DATA_RAM_FUNCTION
10. static bool gpio_check_dlps(uint32_t *timeout)
11. {
12.     return IO_DLPS_Enter_Allowed;
13. }
14.
15. DATA_RAM_FUNCTION
16. static void gpio_enter_dlps(void)
17. {
18.     RCC_PeriphClockCmd(APBPeriph_GPIO, APBPeriph_GPIO_CLOCK, ENABLE);
19.
20.     memcpy((uint32_t *)&gpio_storereg[0], (uint32_t *)&GPIO->DATAOUT, 12);
21.     memcpy((uint32_t *)&gpio_storereg[3], (uint32_t *)&GPIO->INTEN, 16);
22.
23.     gpio_storereg[7] = GPIO->DEBOUNCE;
24.     gpio_storereg[8] = *(__IO uint32_t *) (0x40000344UL);
25.     gpio_storereg[9] = GPIO->INTBOTHEDGE;
26.
27.     /* for evb weakup */
28.     // config this pin (pad) to software mode: waiting ext interrupt to wakeup.
29.     Pad_Config(BUTTON_0, PAD_SW_MODE, PAD_IS_PWRON, PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_LOW);
30.
31.     extern void System_WakeUpDebounceTime(uint8_t time);
32.     System_WakeUpDebounceTime(10);
33.     // use this pin to wakeup the chip from Deep Low Power Mode.
34.     if (GPIO_ReadInputDataBit(GPIO_PIN_BUTTON_0)){
35.         System_WakeUpPinEnable(BUTTON_0, PAD_WAKEUP_POL_LOW, PAD_WK_DEBOUNCE_DISABLE, 0);
36.     }else{
37.         System_WakeUpPinEnable(BUTTON_0, PAD_WAKEUP_POL_HIGH, PAD_WK_DEBOUNCE_DISABLE, 0);
38.     }
39. }
40.
41. DATA_RAM_FUNCTION
42. static void gpio_exit_dlps(void)
43. {
44.     RCC_PeriphClockCmd(APBPeriph_GPIO, APBPeriph_GPIO_CLOCK, ENABLE);

```

```

45.
46. memcpy((uint32_t *)&GPIO->DATAOUT, (uint32_t *)&gpio_storereg[1], 8);
47.
48. memcpy((uint32_t *)&GPIO->INTMASK, (uint32_t *)&gpio_storereg[4], 12);
49.
50. GPIO->DEBOUNCE = gpio_storereg[7];
51. GPIO->DATAOUT = gpio_storereg[0];
52. GPIO->INTCLR = ~(gpio_storereg[1]);
53. GPIO->INTEN = gpio_storereg[3];
54. *(__IO uint32_t *) (0x40000344UL) = gpio_storereg[8];
55. GPIO->INTBOTHEDGE = gpio_storereg[9];
56.
57. /* for evb wakeup */
58. // config this pin (pad) to muxplexing mode.
59. Pad_Config(BUTTON_0, PAD_PINMUX_MODE, PAD_IS_PWRON, PAD_PULL_UP, PAD_OUT_DISABLE, PAD
   _OUT_HIGH);
60. }
61.
62. DATA_RAM_FUNCTION
63. static void gpio_wakeup_dlps(void)
64. {
65.     NVIC_DisableIRQ(System_IRQn);
66.
67.     if (System_WakeUpInterruptValue(BUTTON_0) == SET)
68.     {
69.         Pad_ClearWakeupINTPendingBit(BUTTON_0);
70.         System_WakeUpPinDisable(BUTTON_0);
71.         IO_DLPS_Enter_Allowed = false;
72.     }
73.
74.     // MUST clear debounce bit here.
75.     __rtl_btaon_fast_write_safe_8b(0x2b, (__rtl_btaon_fast_read_safe_8b(0x2b) | BIT7));
76.     NVIC_ClearPendingIRQ(System_IRQn);
77. }
78.
79. static struct dlps_driver_ops rtl_pm_driver_ops[]
80.     __attribute__((section(".dlps.dataon.data")))
81.     __attribute__((__used__)) = {
82.     [0] = {
83.         gpio_check_dlps,
84.         gpio_enter_dlps,
85.         gpio_exit_dlps,
86.         gpio_wakeup_dlps,
87.     },
88. };
89.
90. static int GPIO_INPUT_HANDLER(int irq, void *context, void *arg)
91. {
92.     if (GPIO_GetINTStatus(GPIO_PIN_BUTTON_0)) {
93.         GPIO_INTConfig(GPIO_PIN_BUTTON_0, DISABLE);
94.         GPIO_MaskINTConfig(GPIO_PIN_BUTTON_0, ENABLE);
95.
96.         printk("GPIO_INPUT_HANDLER %d\n", GPIO_ReadInputDataBit(GPIO_PIN_BUTTON_0));
97.         /* Config WakeUp pin */
98.         if (GPIO_ReadInputDataBit(GPIO_PIN_BUTTON_0)){
99.             printk("button release\n");
100.             static struct mible_task_item button_irq_tasklet;
101.             mible_single_task_post(&button_irq_tasklet, button_process_handler, NULL);
102.
103.             GPIO->INTPOLARITY = GPIO->INTPOLARITY & (~GPIO_PIN_BUTTON_0);
104.         }else{
105.             printk("button press\n");
106.             GPIO->INTPOLARITY = (GPIO->INTPOLARITY & (~GPIO_PIN_BUTTON_0)) | GPIO_PIN_
   BUTTON_0;
107.         }
108. #if defined(CONFIG_PM)
109.         IO_DLPS_Enter_Allowed = true;
110. #endif
111.         GPIO_ClearINTPendingBit(GPIO_PIN_BUTTON_0);

```



```

112.     GPIO_MaskINTConfig(GPIO_PIN_BUTTON_0, DISABLE);
113.     GPIO_INTConfig(GPIO_PIN_BUTTON_0, ENABLE);
114. }else{
115.     printk("GPIO_INPUT_HANDLER not BUTTON0\n");
116. }
117.
118.     return 0;
119. }
120.
121. static int board_init(void)
122. {
123.     /* INIT BUTTON IRQ*/
124.     RCC_PeriphClockCmd(APBPeriph_GPIO, APBPeriph_GPIO_CLOCK, ENABLE);
125.
126.     Pad_Config(BUTTON_0, PAD_PINMUX_MODE, PAD_IS_PWRON, PAD_PULL_UP, PAD_OUT_DISABLE,
PAD_OUT_HIGH);
127.     Pinmux_Config(BUTTON_0, DWGPIO);
128.
129.     /* Config WakeUp pin */
130.     System_WakeUpPinEnable(BUTTON_0, PAD_WAKEUP_POL_LOW, PAD_WK_DEBOUNCE_DISABLE, 0);
131.
132.     GPIO_InitTypeDef GPIO_struct_init;
133.     GPIO_StructInit(&GPIO_struct_init);
134.
135.     GPIO_struct_init.GPIO_Pin = GPIO_PIN_BUTTON_0;
136.     GPIO_struct_init.GPIO_Mode = GPIO_Mode_IN;
137.     GPIO_struct_init.GPIO_DebounceTime = 10;
138.     GPIO_struct_init.GPIO_ITTrigger = GPIO_INT_Trigger_EDGE;
139.     GPIO_struct_init.GPIO_ITCmd = ENABLE;
140.     GPIO_struct_init.GPIO_ITDebounce = GPIO_INT_DEBOUNCE_ENABLE;
141.     GPIO_struct_init.GPIO_ITPolarity = GPIO_INT_POLARITY_ACTIVE_LOW;
142.     GPIO_Init(&GPIO_struct_init);
143.
144.     NVIC_InitTypeDef NVIC_InitStruct;
145.     NVIC_InitStruct.NVIC_IRQChannel = GPIO_BUTTON0_IRQn;
146.     NVIC_InitStruct.NVIC_IRQChannelPriority = 3;
147.     NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
148.     NVIC_Init(&NVIC_InitStruct);
149.
150.     GPIO_MaskINTConfig(GPIO_PIN_BUTTON_0, DISABLE);
151.     GPIO_INTConfig(GPIO_PIN_BUTTON_0, ENABLE);
152.
153.     /* register nuttx IRQ*/
154.     extern void up_enable_irq(int irq);
155.     up_enable_irq(GPIO_Group2_VECTORn);
156.     irq_attach(GPIO_Group2_VECTORn, GPIO_INPUT_HANDLER, NULL);
157.
158.     return 0;
159. }

```